

Jess Dowd

CS 499: Computer Science Capstone

Dr. Sherri Maciosek

November 23, 2025

4-2 Milestone Three: Enhancement Two: Algorithms and Data Structure Narrative

For my second enhancement I have improved the Travlr gateways website artifact. I built this project for CS 465 in 2025. It uses MEAN stack, which is (Mongo, Express, Angular, Node). It shows trips and info for vacationing, and allows editing of the trips. It has a RESTful API that allows CRUD operations through HTTP methods to handle data requests between the client and server.

I selected this artifact for enhancement because I want to demonstrate my skills in web development, and backend security with the MEAN stack. I specifically want to show off my skills in at expanding the complexity of the use of data structures and algorithms in a MEAN stack project, creating and working with interfaces, following good security practices, working with OOP, data and algorithms, RESTful API, and my ability to work with several different languages, as well as my skills working with github, VSC, Bash, Postman, which are important tools and technologies.

For this enhancement I have added and improved several features with clean, easily readable and secure code. Previously any user account could edit trip info and had access to full admin privileges, to fix this I have expanded the complexity and use of data structures and

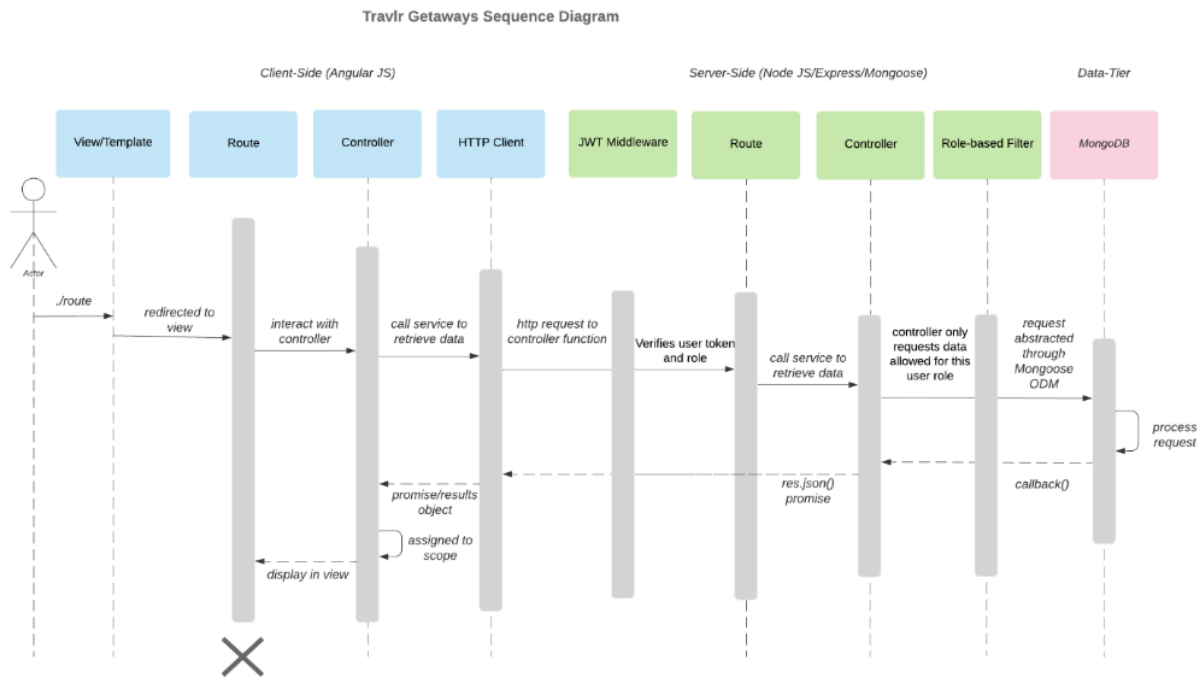
algorithms in my project by adding an RBAC (role-based access control) authentication system with MongoDB with separate user types that has different role-based permissions.

Before anyone could see the trip info without being logged in. So, to fix this I have also hidden the trip info for non-logged in users in addition to implementing the RBAC enhancement. I cleaned the code by removing any unused and duplicate old code, as well as improving comment and structure of the code.

To implement these enhancements, I have expanded the data model by introducing new fields in the User schema, and I restructured the authentication logic and added middleware for permissions. I used hashing for passwords and conditional logic for access control. The Express middleware can now verify the JWT token and check the user's role before allowing access to protected routes such as editing trip information.

I enhanced data structures such as user documents, role fields and object relationships as well as improving and building algorithmic with control-flow checks, conditional authorization logic and token verification.

As shown in my sequence diagram below, my enhancement now routes all protected HTTP client requests through a JWT middleware layer. This middleware verifies the user's token first, and only allows the request to reach the server if the user passes the role-based filter.



Logged in as viewer account type:

The screenshot shows the TravlR web application interface. The browser address bar indicates the URL is `localhost:4200`. The page header includes the TravlR logo and a "Trips Log Out" link. The main content area displays a list of travel packages, each with a title, a thumbnail image, a price, and a description. The packages are:

- Gale Reef (updated)**: Gale Resort, 4 days only \$399.00 per person. Description: "Gale Reef Sed et augue lorem. In sit amet placerat arcu. Mauris volutpat ipsum ac justo mollis vel vestibulum orci gravida. Sed consequat libero ut turpis venenatis ut aliquam risus semper. Etiam convallis mi vel risus pretium sodales."
- Dawson's Reef**: Dawson Resort, 5 days only \$499.00 per person. Description: "Dawson's Reef Integer magna leo, posuere et dignissim vitae, porttitor at odio. Pellentesque a metus nec magna placerat volutpat. Vulputate, vulputate at est. Morbi aliquet viverra metus eu consectetur. In lorem dui, elementum sit amet convallis."
- Claire's Reef**: Claire Resort, 8 days only \$299.00 per person. Description: "Claire's Reef Donec sed felis risus. Nulla facilisi. Donec a orci tellus, et auctor odio. Fusce ac orci nibh, quis semper arcu. Vulputate eget libero. Integer purus justo, egestas eu consectetur eu, cursus in tortor. Quisque nec nunc ac mi."
- Test Trip 4**: Test Resort, 5 days only \$440.00 per person. Description: "A test trip card."

Logged in as admin account type:

← → ↺ ⓘ localhost:4200 🔑 🔍 ☆ 🏠 📁 🖼️ 👤 ⋮

Travel Trips Log Out

Add Trip

Gale Reef (updated)



Gale Resort

4 days only \$399.00 per person

Gale Reef Sed et augue lorem. In sit amet placerat arcu. Mauris volutpat ipsum ac justo mollis vel vestibulum orci gravida.

Sed consequat libero ut turpis venenatis ut aliquam risus semper. Etiam convallis mi vel risus pretium sodales.

Edit Trip

Dawson's Reef



Dawson Resort

5 days only \$499.00 per person

Dawson's Reef Integer magna leo, posuere et dignissim vitae, porttitor at odio. Pellentesque a metus nec magna placerat volutpat.

Vulputate, vulputate at est. Morbi aliquet viverra metus eu consectetur. In lorem dui, elementum sit amet convallis.

Edit Trip

Claire's Reef



Claire Resort

8 days only \$299.00 per person

Claire's Reef Donec sed felis risus. Nulla facilisi. Donec a orci tellus, et auctor odio. Fusce ac orci nibh, quis semper arcu.

Vulputate eget libero. Integer purus justo, egestas eu consectetur eu, cursus in tortor. Quisque nec nunc ac mi.

Edit Trip

Test Trip 4



Test Resort

5 days only \$440.00 per person

A test trip card.

Edit Trip

Not logged in:

← → ↺ ⓘ localhost:4200 🔑 🔍 ☆ 🏠 📁 🖼️ 👤 ⋮

Travel Trips Log In

Please log in to view and manage trips.

Log in

Connected handlebars template to trips page to display updates

The trips info being called and received from the database was not displaying in the front-end localhost 3000 main website interface on the frontend, that was something I had not implemented for the original project, but I wanted to complete to make it a true functioning full stack application. I had to figure out what was causing the disruption of the data, so I set up a test page to check the status of API at my frontend at localhost3000. The results showed me that the front end was receiving the api data properly, I knew it wasn't that. That told me it was likely in app_server because the issue is with the data being displayed. To fix it, I checked the relevant files in app_server for any errors, I found that I had forgotten to implement the new HTML layout in handlebars for the travel page. I replaced the old html blocks with the new Handlebars layout and helpers and relaunched the server to test.

```

{{!-- Render each trip returned by app_server/controllers/travel.js --}}
{{#each trips}}
  <li>
    <!-- Trip image and name -->
    <a href="/travel"></a>
    <h2><a href="/travel">{{name}}</a></h2>
    <!-- Trip description allows HTML from the API -->
    <div class="trip-description">
      {{{description}}}
    </div>
  </li>
{{/each}}
```

localhost:3000/api/trips

pretty-print ☒

```
{
  "_id": "68df739fefac0758cd49da71",
  "code": "GALE-REEF",
  "name": "Gale Reef (updated)",
  "image": "reef1.jpg",
  "resort": "Gale Resort",
  "length": "4 days",
  "perPerson": 399,
  "description": "\u003Cp\u003EGale Reef Sed et augue lorem. In sit ame
},
```

Hide/Show trips based on logged-in or not

First I updated the component class to expose isLoggedIn(). In trip-listing.ts I added a new import.

Then I encapsulated the trip-listing template with a *ngIf which will hide the entire template if not logged in, and will instead display a new logged out view template with a more prominent login button and a message prompting the user to login.

```
<div *ngIf="isLoggedIn(); else loggedOut" class="container my-3">
```

Please log in to view and manage trips.

Log in

Added role to the user model in Express

Previously I only had email, name, hash, salt for the user model parameters, to implement the enhancement I added in a new parameter in the user schema that defines user types of admin and viewer, and sets the default account type to viewer, so that any accounts that have been previously created or that do not specify account type will default to viewer. I then pushed the new role from Mongo into the JWT

```
// New user role
role: {
  type: String,
  enum: ['admin', 'viewer'],
  default: 'viewer'
};
```

```
userSchema.methods.generateJWT = function(){
  return jwt.sign(
    { _id: this._id, email: this.email, name: this.name, role: this.role },
```

I made sure registration will set the role in express, so I modified register in authentication.js so that it accepts an optional role but will default to viewer for security purposes.

```
// Create user document
const user = new User({
  email: email.toLowerCase().trim(),
  name
});
// Enhancement 2: assign role (default viewer)
user.role = req.body.role || 'viewer';
```

I added the requireRole() middleware in index.js. It runs after the JWT authentication, and the decoded token is attached to req.user, then the middleware checks the role, if it comes back with an issue it will give a 403 status message clarifying the issue and stating either there is no role in the token, or that the role is forbidden for that action.

```
// Role-based access check (Enhancement 2)
// Blocks requests unless the user's JWT contains the required role.
// - Used for admin-only routes (trip writes, user management)
// -----
function requireRole(role) {
  return function (req, res, next) {
    if (!req.auth || !req.auth.role) {
      return res.status(403).json({ message: 'No role in token' });
    }
    if (req.auth.role !== role) {
      return res.status(403).json({ message: 'Forbidden: role ' + req.auth.role + ' cannot do this' });
    }
    next();
  };
}
```

I then protected the write operations (POST (create), and PUT (update)) so that to make any changes the user needs to have JWT and also be an admin role. Now the backend is complete.

```
// Trip routes (Enhancement 2 protects writes)
// Public: read-only (GET)
// Admin only: create/update (POST, PUT) when JWT + role = admin
// -----
router.get('/trips', ctrlTrips.listTrips);
router.get('/trips/:tripCode', ctrlTrips.readTrip);

router.post('/trips', authenticateJWT, requireRole('admin'), ctrlTrips.createTrip);
router.put('/trips/:tripCode', authenticateJWT, requireRole('admin'), ctrlTrips.tripsUpdateTrip);
```

Exposed the user role to SPA in Angular

I updated the SPA so it can grab role along with the other user data and I updated `getCurrentUser` in `authentication.ts` to accept a token with the new role payload.

```
// token now has: _id, email, name, role
return {
  email: payload.email,
  name: payload.name,
  role: payload.role || 'viewer'
};
```

Hide/Show buttons based on role

Next I used `*ngIf="isAdmin()"` for the add trip button in the HTML, which will hide the button if not a admin user type. And in the component, I added a public boolean function to check the auth if the user is an admin type or not.

I repeated the same process with the edit trip button, I added `*ngIf="isAdmin()"` for the button contained in the trip cards, and edited the component to check the authentication service for admin user type before showing the button to the user.

```
<button class="btn btn-primary mb-3" *ngIf="isAdmin()" (click)="addTrip()">Add Trip</button>
```

```
public isAdmin(): boolean {
  return this.auth.isAdmin();
}
```

I met several course outcomes with this enhancement. I met the first outcome because this enhancement supports a collaborative environment to support decision making, by having a feature that supports different user types to participate in the use of the product. In a real-world project RBAC is important for organizational decision making because it controls who can modify data and who can view it. This feature would be able to support organizational decision making through the use of view-only info for some accounts such as different employee levels or customers.

I demonstrated the third course outcome by designing and implementing a solution that solves a important problem in web development which is data and software security, and I used computer science best practices in full stack development to implement the enhancement.

I implemented the well-founded technique of RBAC through MongoDB and Express which demonstrates practical industry-level skills that are commonly used in web applications and professional backend development, this shows proficiency in the fourth course outcome.

I demonstrated the fifth course outcome by showing a security mindset by implementing this enhancement which strengthens the overall architecture by adding proactive security layers that stop any unauthorized access, this shows I have understanding of both prevention and design-based mitigation.

During this enhancement I further improved my skills in algorithms, database design, and good security practices, including password and token-based verification. I used hashing for passwords and conditional logic for access control in a MEAN stack web environment. As I was working on this project, I improved my skill at integrating data structures and algorithmic

thinking to add a feature and also improve security, which aligns with my career goals and course outcomes.

I faced some challenges along the way. One major thing was debugging the data not displaying on the front-end. I had to trace the issue from the client to the server, test API responses in Postman, and eventually I found out that the missing Handlebars template was preventing the trip data from rendering. Fixing this required me to use troubleshooting skills and full-stack reasoning.